

Guided Soft Actor–Critic

Blind Go2-W Locomotion — Theory and Implementation

Tamer Bora İkizoğlu

Istanbul Technical University

Outline

The problem

Guided SAC: the idea

Architecture

Objectives

Schedules

Deployment

Implementation

The problem

Blind locomotion is a POMDP

In simulation we can read anything: body linear velocity, a height scan of the terrain, contact forces, friction, every randomized dynamics parameter.

On the real robot almost none of that exists. The deployed policy sees only proprioception: joint encoders, IMU, its own previous action.

The asymmetry

Training-time information $\gg$ deployment-time information. A policy trained on what it *cannot* have at deployment is useless; a policy that ignores what the simulator knows is wasteful.

Guided SAC is one answer: let a *privileged* network help train a *blind* network that is the only thing shipped.

Two observation channels



Proprioceptive, noisy, 5-frame history

5×57 : base ang. vel. (3), projected gravity (3), velocity command (3), joint pos (16), joint vel (16), previous action (16)

Privileged, clean, single frame

$57 + 3 + 187$: same frame without noise, base *linear* velocity (3), height scan (187)

Action space: 16 dims = 12 leg position targets + 4 wheel velocity targets, tanh-squashed and scaled by per-dimension bounds (legs 5.0, wheels 10.0).

Guided SAC: the idea

Origin, and what we changed

Haklıdır & Temeltaş (2021) and Çerçi & Temeltaş (2024) train **two actors**: a *guiding actor* that sees clean state, and a *control actor* that sees degraded state and is pulled toward the guide's actions.

	Çerçi (2024)	This project
Why the control actor is handicapped	adversarial perturbation	<i>genuine</i> partial observability
Guiding actor sees	clean state s_t	privileged $\tilde{s}_t$ (height scan, lin. vel.)
Control actor sees	perturbed s_t^p	proprioception o_t + history
Shapes	identical	different (285 vs 532)
Agents	$k = 3, 5, 7$	$k = 1$ (single agent)

Two consequences: the paper's $s_t^p \rightarrow o_t$ mapping is *not* cosmetic (different dimensionality), and every multi-agent construction collapses at $k = 1$ — it must be re-derived, not copied.

Guided SAC is three mechanisms, not one

M1 — Asymmetric critic

$Q(\tilde{s}_t, a)$ sees everything,
 $\pi_f(a|o_t)$ does not

cost: zero extra networks
fixes credit assignment

M2 — Guide as behavior policy

the teacher also *acts*
in the environment

cost: one extra actor
widens state visitation

M3 — Action-space imitation

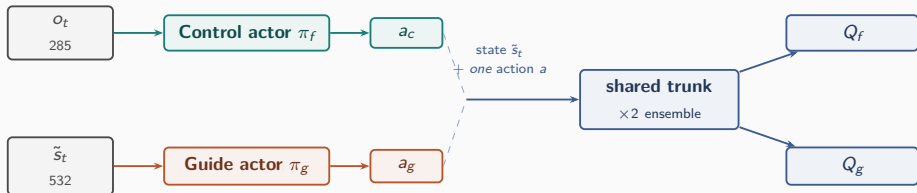
pull π_f toward π_g
with weight λ

cost: one norm term
dense low-variance target

The literature ships these **bundled**. Separating them matters: M1 is free and not specific to Guided SAC, while M2 is the expensive one. Our design keeps M1, keeps M3 in softened form, and uses only a *small* amount of M2 — for a different reason than the original (next slides).

Architecture

Three networks, and two value functions



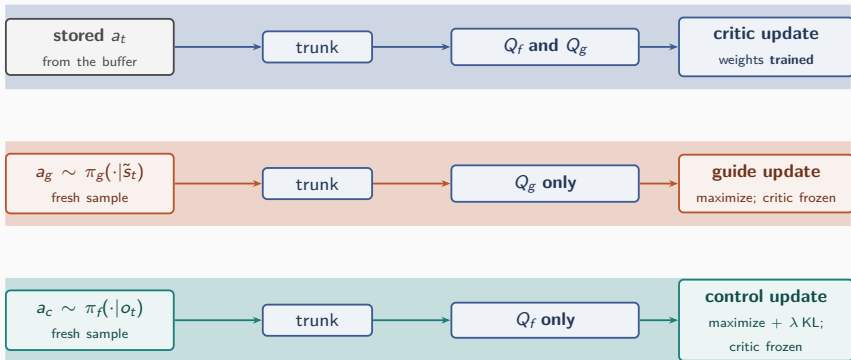
What the two heads mean

$Q_f(\tilde{S}_t, a)$ — how good is taking action a now, **if the blind student acts from then on**

$Q_g(\tilde{S}_t, a)$ — how good is taking action a now, **if the privileged teacher acts from then on**

Same body, same input; only the output heads differ. *Which* action is fed in — and which head answers — depends on the update (next slide).

Which action reaches which head

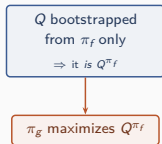


The trunk is the *same* network in all three lanes. Two rules make the whole design readable: the critic is **only ever trained on actions that were actually executed**, and each actor **only ever reads its own head-pair** — so the student optimizes the value of the deployed policy and never the teacher's.

Why two value functions, and not one

One head-pair per actor: π_f reads Q_f , π_g reads Q_g .

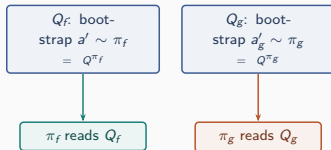
One shared critic (naive)



one greedy improvement
step over the student;
improvements never re-enter the
bootstrap $\Rightarrow$ they never compound

The teacher's advantage is **capped by construction** — contradicting the premise that a fully-observed teacher is better.

Two head-pairs (ours)



Each actor optimizes against *its own* fixed point, so the teacher can run ahead — while the student still optimizes the **deployed** policy's value.

This recovers, in twin- Q form, the role of Çerçi's separate V network.

Objectives

How the two value functions are trained

	Q_f (student head)	Q_g (guide head)
trained on	executed action a_t	executed action a_t
“and then?”	the student π_f continues	the teacher π_g continues
temperature used	α_f	α_g
so it estimates	Q^{π_f}	Q^{π_g}

Each head uses the ordinary SAC target — *the reward that really followed, plus the discounted value of the next state* — and only the policy used to value that next state differs. Distributional critic, n -step returns and the EMA target network are inherited unchanged.

Actors: one new term in the whole method

The guide is **plain SAC**. The student is **plain SAC plus one pull toward the guide** — that single term is the entire method.

Guide — ordinary SAC on privileged input, against its own heads:

$$\mathcal{L}_{\pi_g} = \mathbb{E} \left[\alpha_g \log \pi_g(a_g | \tilde{s}_t) - \min_i Q_{g,i}(\tilde{s}_t, a_g) \right]$$

Control — the same objective on proprioception, plus guidance:

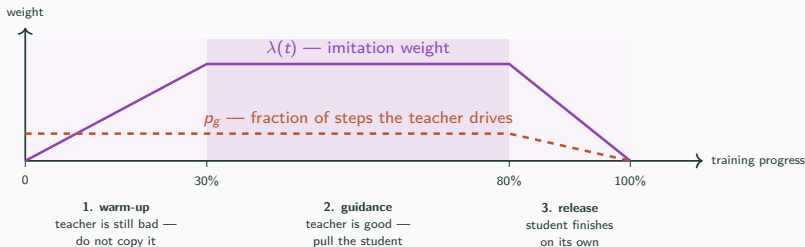
$$\mathcal{L}_{\pi_f} = \mathbb{E} \left[\alpha_f \log \pi_f(a_c | o_t) - \min_i Q_{f,i}(\tilde{s}_t, a_c) + \underbrace{\lambda(t) \bar{q} D_{\text{KL}}(\pi_f(\cdot | o_t) \parallel \text{sg}[\pi_g(\cdot | \tilde{s}_t)])}_{\text{the only term Guided SAC adds}} \right]$$

- **KL, not L_2 .** Both policies are tanh-Gaussian, so the KL is closed-form and matches the teacher's *variance*, not just its mean — the student may stay uncertain where the teacher is. The teacher is stop-gradded.
- $\bar{q} = \mathbb{E}[\min_i Q_{f,i}]$ keeps λ dimensionless as values grow; α_f, α_g are learned **separately**.

Schedules

λ : how hard the student is pulled toward the teacher

λ is the **weight on the imitation term**: $\lambda = 0$ means the student ignores the teacher entirely (plain SAC), larger λ pulls it harder toward copying the teacher. It is **not constant**:



p_g is a *different* knob — the small fraction of steps the teacher actually drives — so that Q_g is trained on actions the teacher really took.

The imitation gap — the method's real weakness

π_g solves the *fully observed* MDP. π_f must solve the *POMDP*. These are different problems, and projecting one onto the other is not optimal in general.

Concretely, for blind locomotion

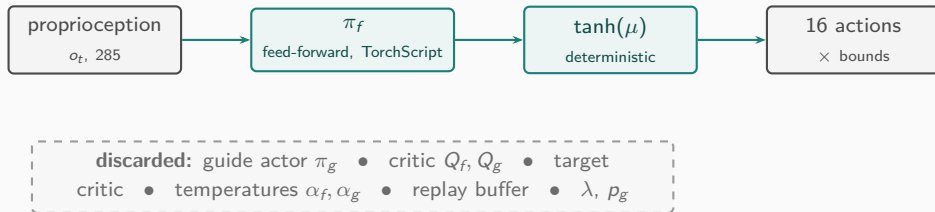
A teacher that sees the height scan never learns to probe the ground with a foot, never steps cautiously into the unknown, never moves to enrich its own IMU signal — it has no need to. Imitating it can therefore *suppress exactly the behaviours that keep a blind robot alive*.

Mitigations built in, weakest to strongest:

1. **KL rather than L_2** — the student may stay uncertain where the teacher is.
2. $\lambda \rightarrow 0$ — guidance is removed before training ends.
3. **Latent supervision** (planned): supervise a learned *representation* $\hat{z} = E(o_t)$ against privileged z and let the policy decide how to use it — closes the gap by construction.

Deployment

Deployment — what survives



- The deployed artifact is **architecturally identical** to a vanilla SAC actor. Guided training costs *nothing* at inference — the strongest practical argument for asymmetric methods.
- Nothing privileged can leak: the actor consumes a fixed 285-dim slice and the export path never reads the guide's files.

Implementation

Code map

Built as an **extension** of the FlashSAC backbone, never a fork: the vendored learning core is byte-identical, and without the `--guided` flag the pipeline is the unmodified baseline.

<code>gsac/config.py</code>	config dataclass; $\lambda(t)$, $p_g(t)$; fail-fast validation
<code>gsac/network.py</code>	GuidedDoubleCritic: shared trunks + second head-pair
<code>gsac/update.py</code>	the three losses above, each traced to its source equation
<code>gsac/agent.py</code>	GuidedFlashSACAgent: networks, rollout mixing, updates

<code>train.py --guided</code>	launcher flag; everything guided is behind it
--------------------------------	---

The subclass overrides `sample_actions` (mixing), `update` (the three losses) and `save/load` (guide checkpoints); everything else is inherited. The teacher's action a_g is never stored — it is recomputed from the sampled batch at update time, so teacher/state alignment holds by construction.

One codebase, four experiments

Each rung switches on *one* mechanism, so any difference in outcome can be attributed to it:

	critic sees	guide trained?	guide acts?	imitation	
A1	o_t (blind)	no	no	no	vanilla SAC
A2	$\tilde{s}_t$ (privileged)	no	no	no	+ M1
A4	$\tilde{s}_t$	yes	$p_g > 0$	no ($\lambda = 0$)	+ M2
A5	$\tilde{s}_t$	yes	$p_g > 0$	yes ($\lambda > 0$)	+ M3 = full Guided SAC

A1 → **A2** is not “no teacher vs teacher” — neither has a second network. It is purely what the *critic* may see: vanilla SAC trains a blind critic too, while M1 hands it the height scan and leaves the actor blind. A teacher appears only at A4, and only *influences* the student at A5.

Which mechanism actually pays — and is the second actor worth its cost? The literature has never separated them.

Summary

- Blind locomotion is a genuine POMDP; the simulator knows far more than the robot ever will.
- Guided SAC bundles three separable mechanisms. We keep the free one (asymmetric critic), soften the risky one (imitation via scheduled KL), and use the expensive one sparingly and for a different reason than the original (value grounding).
- The single-agent reduction is not a copy: the teacher needs **its own value head**, otherwise its advantage is capped by construction.
- The imitation gap is the method's real theoretical weakness; latent supervision is the principled successor if action-space imitation does not pay.
- **Deployment cost is exactly zero** — the shipped network is a plain SAC actor on proprioception.